

SOUTENANCE DE PROJET DE FIN D'ANNÉE



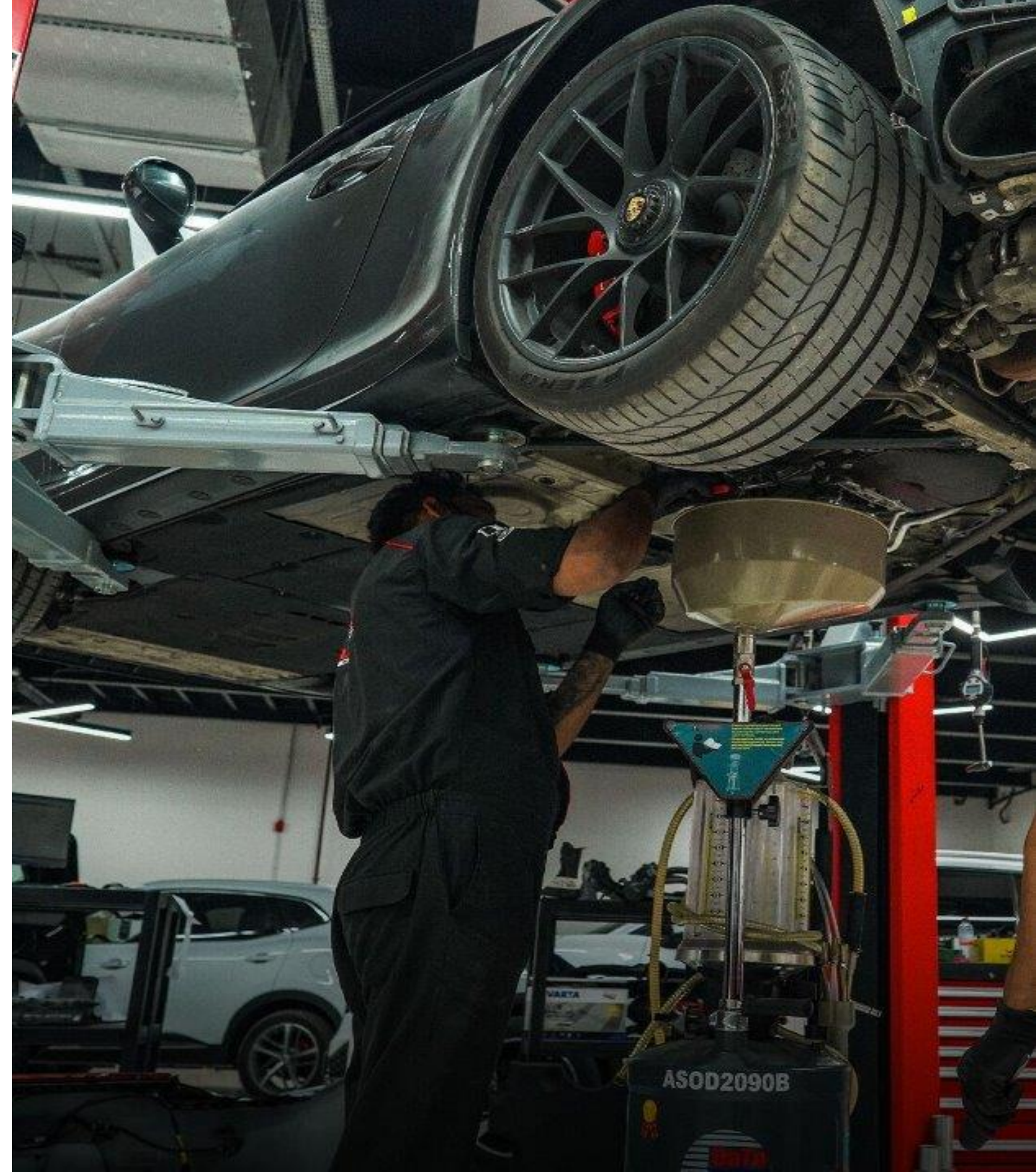
Plateforme e-commerce automobile · huiles, pièces & vidange à domicile

ÉQUIPE

- BAKHTAOUI Mohammed
- BOUROUGA Mohammed Amine
- EL GHALI Achraf
- AZAMRI Ayoub
- EL OUALI Abdessamad

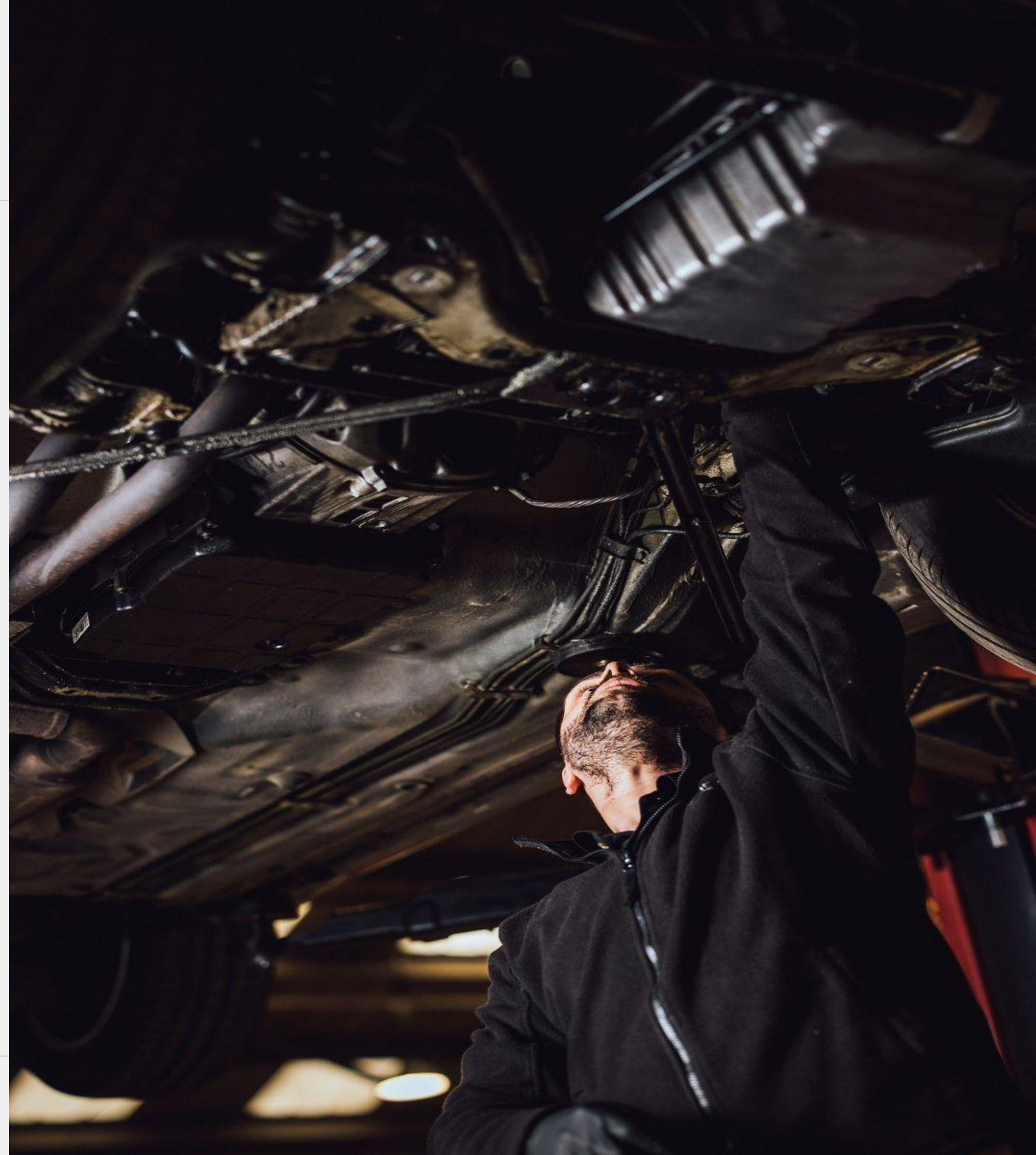
ENCADRANT

M. MANI Mohammed Adil



PLAN

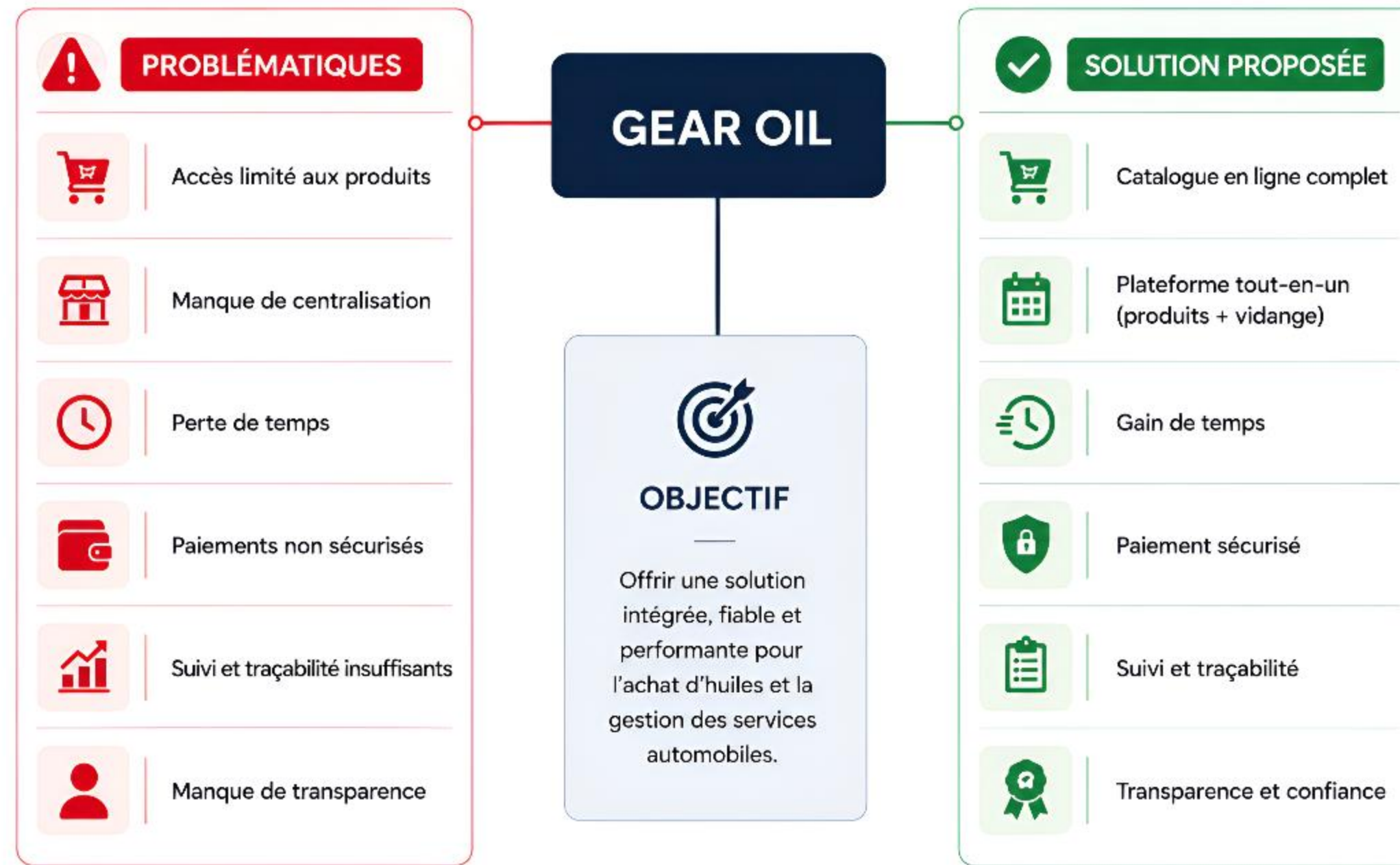
- 01** INTRODUCTION
- 02** PROBLEMATIQUES ET SOLUTIONS
- 03** DESCRIPTION DES BESOINS
- 04** ANALYSE & MODÉLISATION
- 05** RÉALISATION
- 06** CONCLUSION & PERSPECTIVE



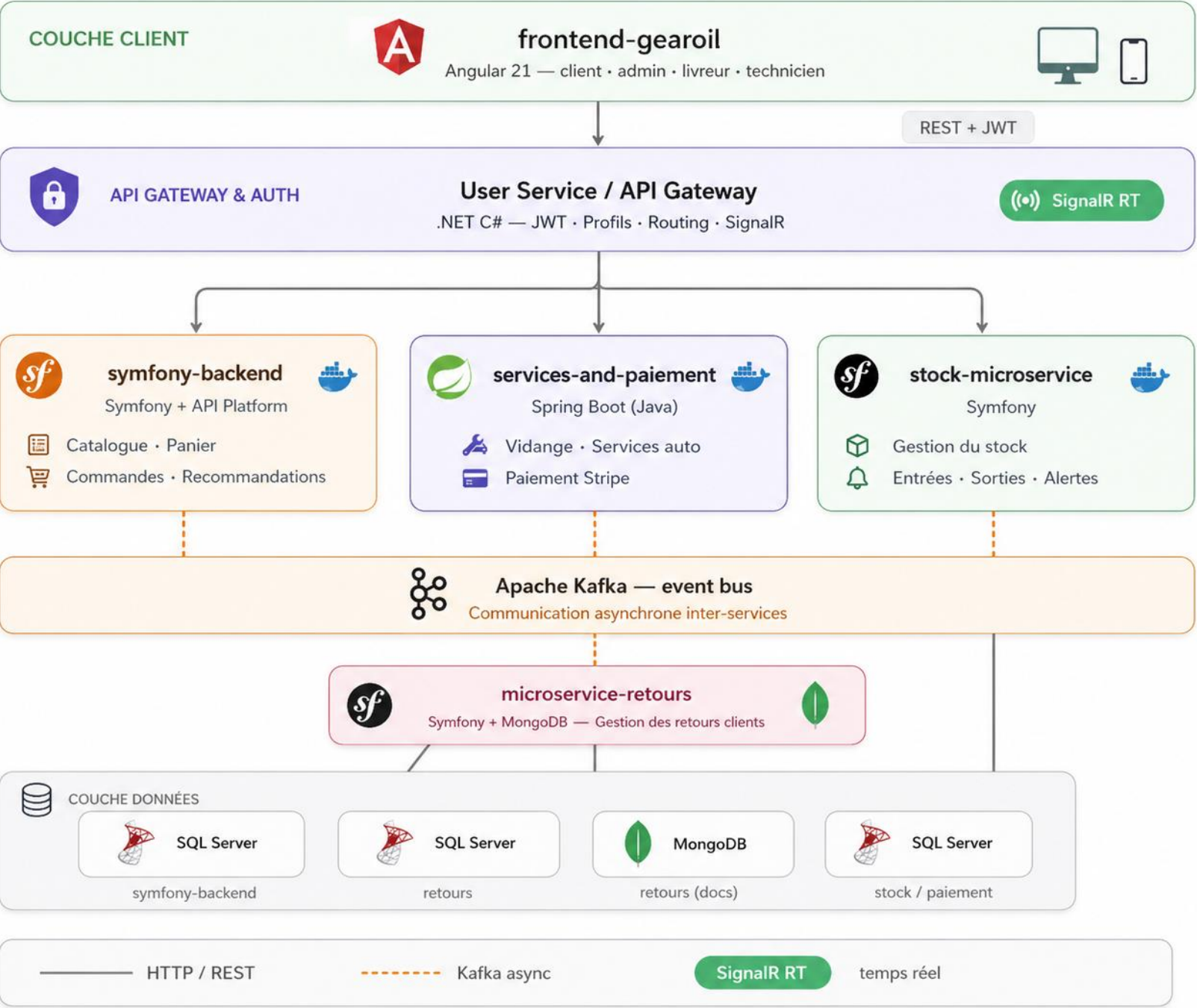
Introduction



Problématique & *Solution*



Solution *Globale*



01 · FRONTEND

Angular 21

SPA + SSR · Tailwind 3.4 · Leaflet · Swiper

02 · IDENTITY · GATEWAY

.NET 9 + Ocelot 24.1

JWT · API Gateway · EF Core 9 + SQL Server

03 · MICROSERVICES

Spring Boot · Symfony

Stripe · Twilio · MongoDB · Redis · Kafka

03

Description *des Besoins*

Besoins *Fonctionnels*

01



Gestion utilisateurs

- Inscription / Connexion
- Gestion des rôles
- Profils & permissions

02



Catalogue de produits

- Catalogue intelligent
- Stock temps réel
- **Recommandations IA** basées sur le comportement, l'historique d'achats et la compatibilité véhicule

↑IA RECOMMANDÉE

03



Panier & commandes

- Panier
- Validation commande
- Suivi en temps réel
- Synchronisation du stock

04



Paieement sécurisé

- Paiement via Stripe
- Facture automatique
- Remboursement

05



Livraison

- Suivi en temps réel
- Notifications SMS / Email
- Preuve de livraison
- Historique des livraisons

06



Notifications

- Notifications SMS / Email
- Alertes en temps réel
- Rappels automatiques

Besoins *Non Fonctionnels*



PERFORMANCE

Temps de réponse · requêtes critiques



Redis Cache

Données fréquemment demandées en mémoire



Indexation BDD

Index sur les champs les plus utilisés



Pagination

Affichage par pages pour limiter la charge



Lazy Loading

Composants chargés à la demande (Angular)



FIABILITÉ

Disponibilité · sauvegarde automatique



Transactional Outbox

Événements garantis (at-least-once)



Idempotence

Évite les doublons en cas de rejeu d'événements



Rollback

Transactions ACID pour la cohérence des données



Gestion concurrence

Verrouillage lors des opérations critiques (ex: stock)



SCALABILITÉ

Multi-users simultanés · nouveaux ms



Microservices

Services indépendants et découplés



Kafka

Communication asynchrone et résiliente



API Gateway

Point d'entrée unique, routage & sécurité



MAINTENABILITÉ

Microservices modulaires · séparés



SOLID

Principes de conception respectés



Design Patterns

Architecture propre et évolutive



Code modulaire

Facile à maintenir et à faire évoluer



SÉCURITÉ

JWT + bcrypt · HTTPS · Stripe



JWT Auth

Tokens sécurisés (access & refresh)



HTTPS / TLS

Chiffrement des communications client & services



Stripe

Paielements sécurisés, et conformes PCI-DSS



PORTABILITÉ

Docker · déploiement identique



Docker

Conteneurs isolés et reproductibles



Variables .env

Configuration externalisée et sécurisée



Déploiement simplifié

Même environnement dev → prod



ACCESSIBILITÉ

Ergonomie · clients, admins, techs



Interfaces par rôle

Adaptées à chaque type d'utilisateur



Navigation simplifiée

Parcours utilisateur intuitif



UX fluide

Expérience rapide et agréable



IA / UX

Recommandations IA + push



Recombee

Recommandations intelligentes basées sur le comportement



Notifications temps réel

Push, SMS, Email pour chaque événement

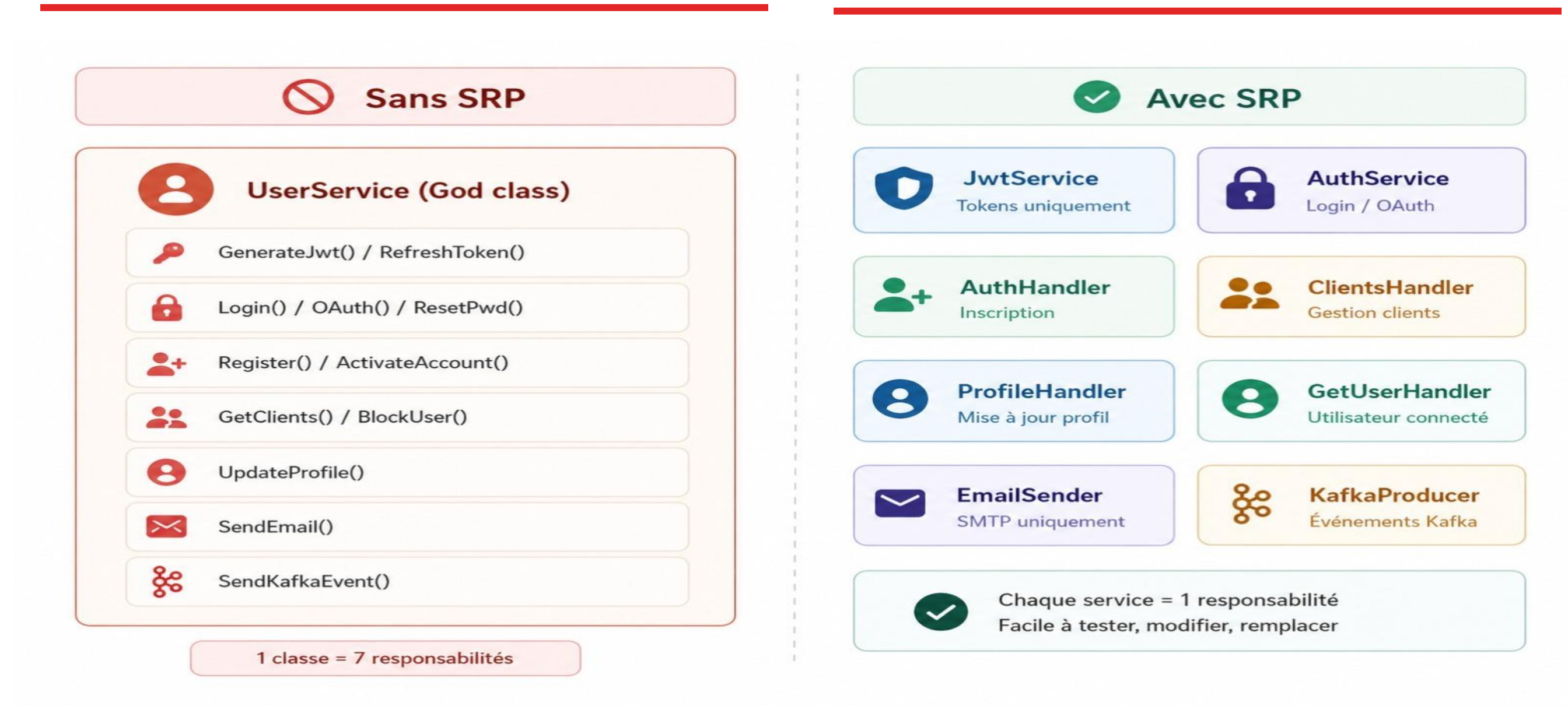


Personnalisation

Expérience adaptée à chaque utilisateur

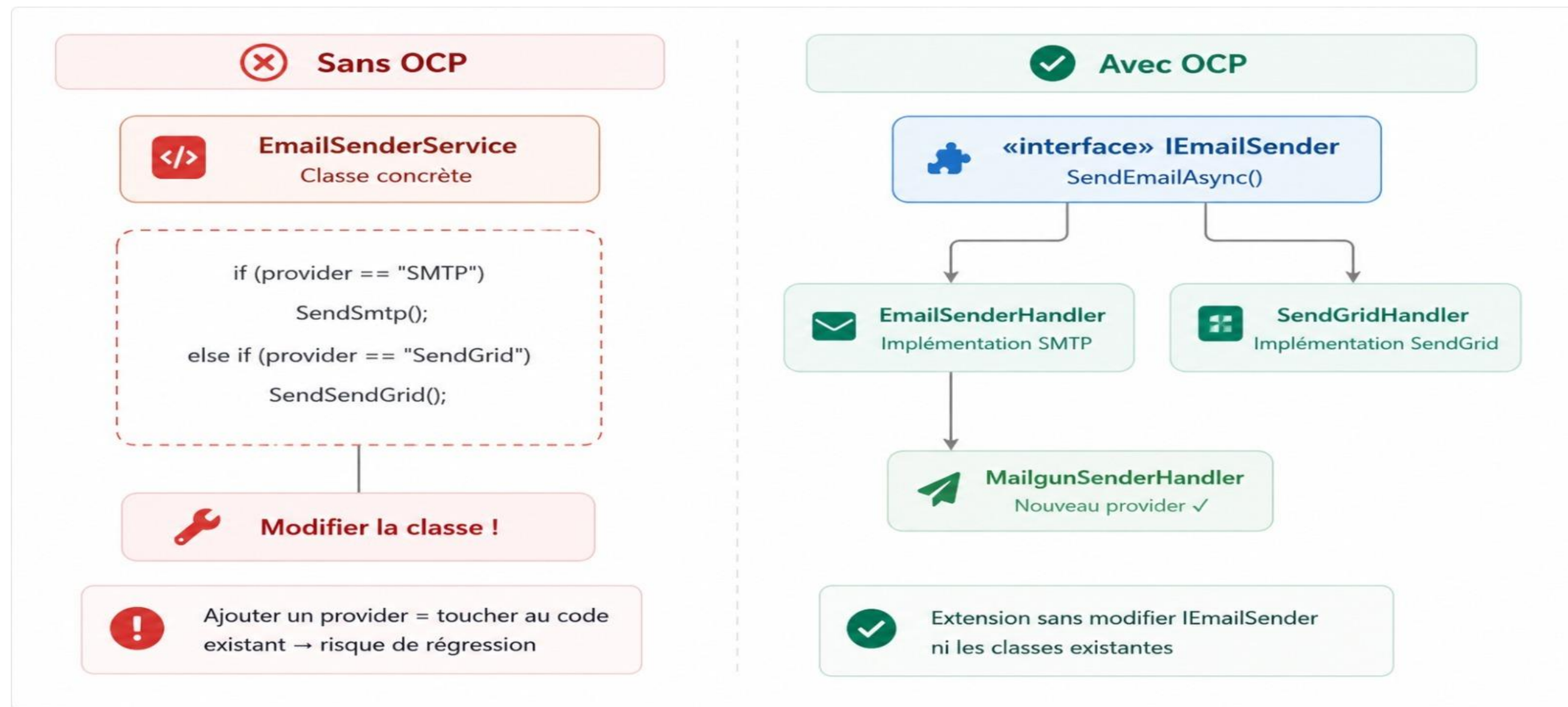
Besoins *Non Fonctionnels*(Principe SOLID)

S — Single Responsibility Principle : sans SRP, tout est dans une seule "God class" difficile à tester et à maintenir. Avec SRP, chaque service n'a qu'une seule raison de changer.



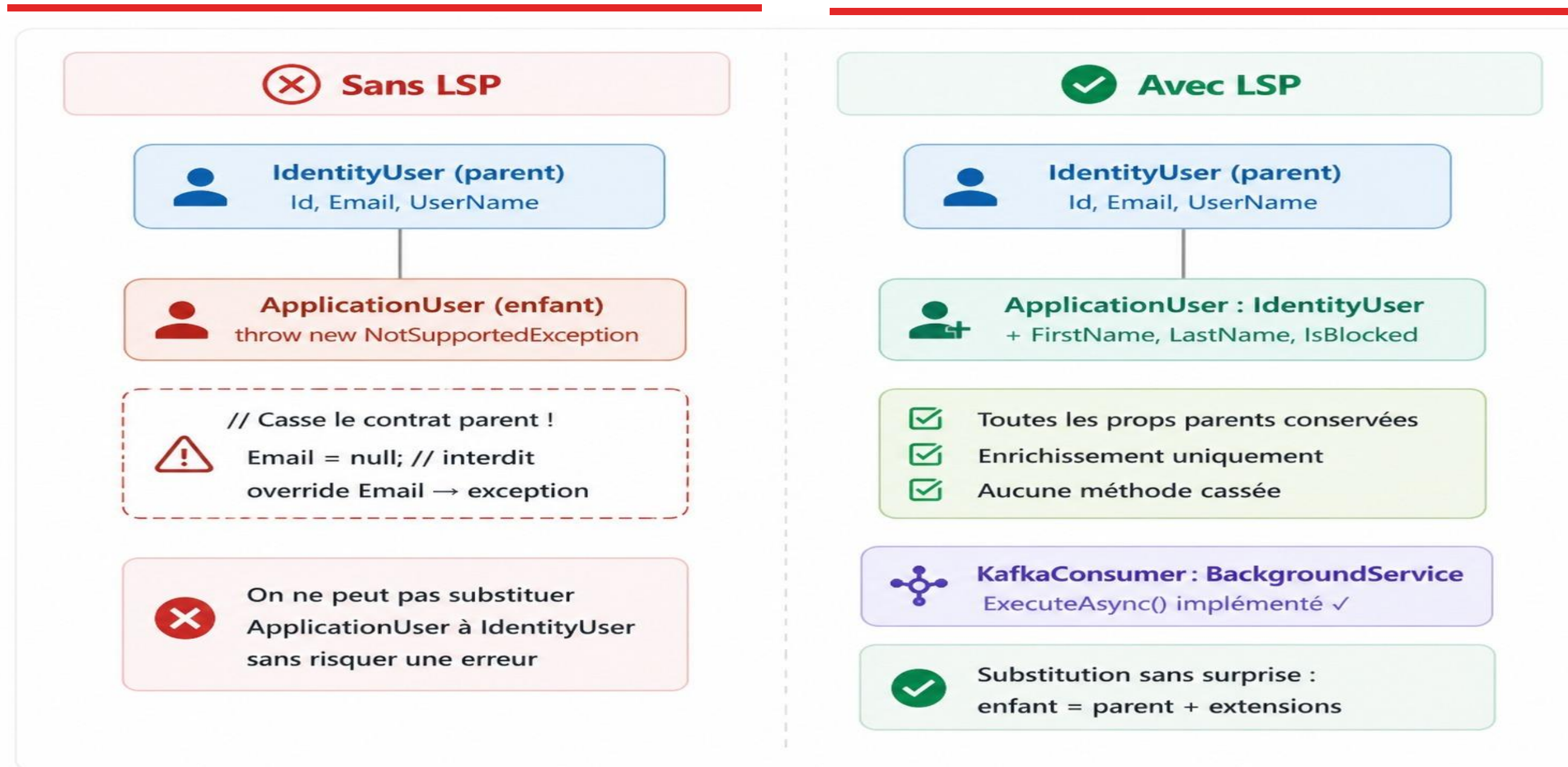
Besoins *Non Fonctionnels* (Principe *SOLID*)

O — Open/Closed Principle : sans OCP, ajouter SendGrid force à modifier EmailSenderService (risque de bugs). Avec OCP, on crée juste une nouvelle implémentation de IEmailSender.



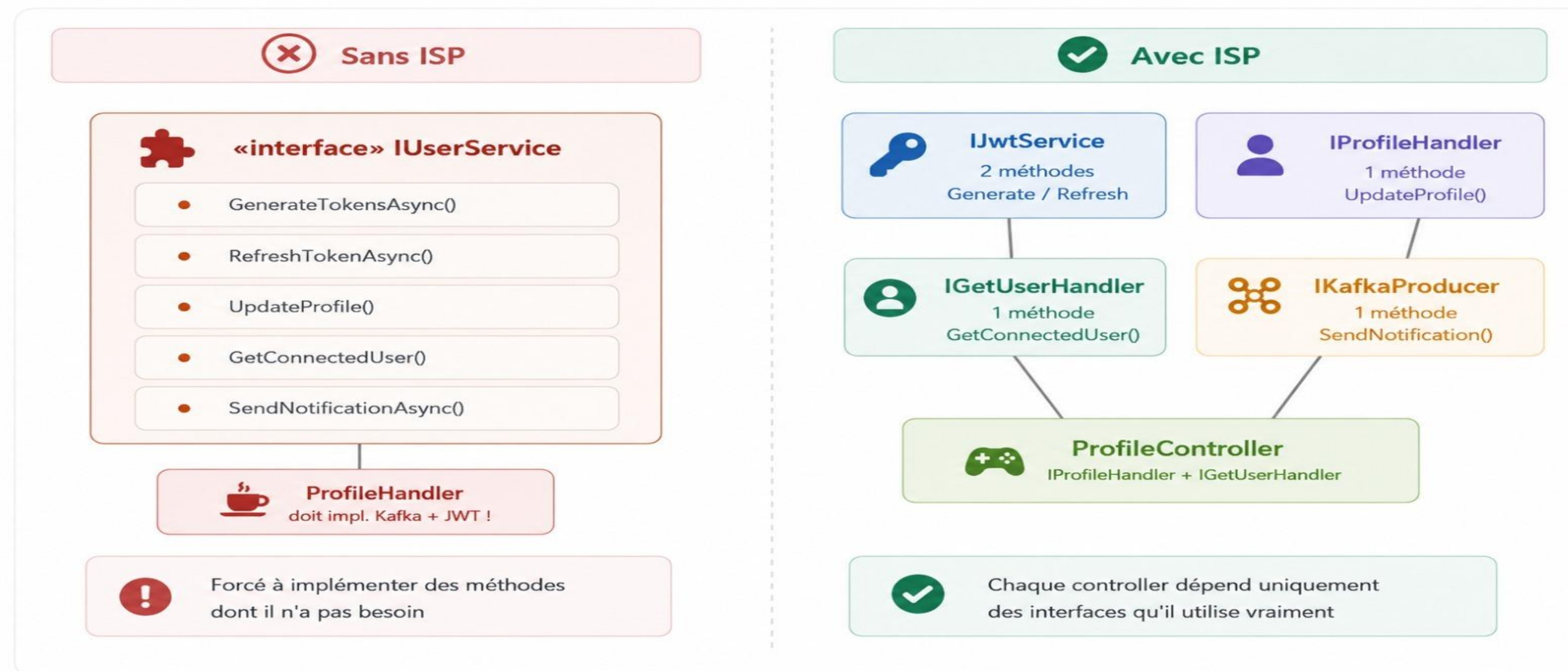
Besoins *Non Fonctionnels* (Principe *SOLID*)

L — Liskov Substitution : sans LSP, l'enfant casse le contrat du parent (exception sur des propriétés attendues). Dans ton projet, ApplicationUser n'écrase rien d'IdentityUser, il enrichit seulement.



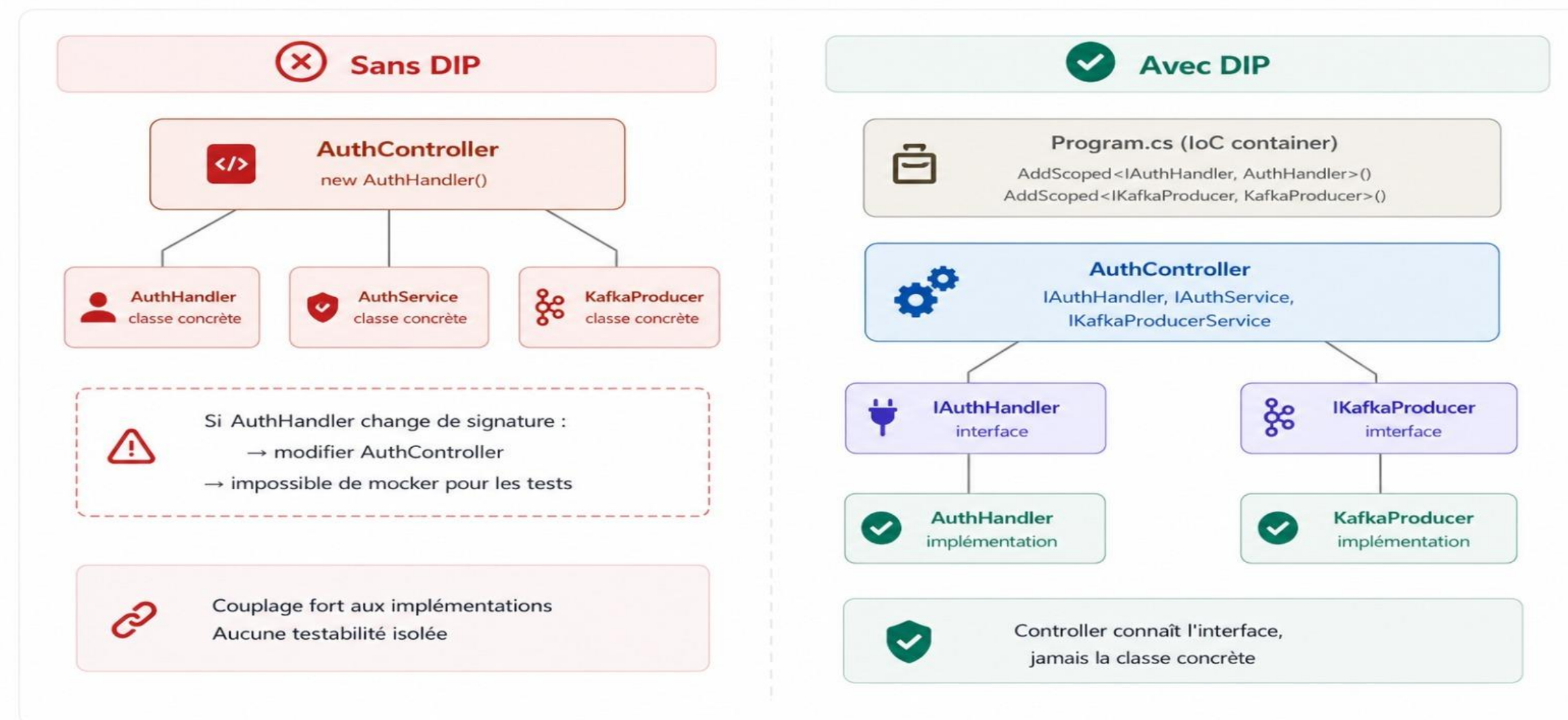
Besoins *Non Fonctionnels* (Principe *SOLID*)

I — Interface Segregation : sans ISP, une seule interface géante force ProfileHandler à implémenter Kafka et JWT dont il n'a rien à faire. Avec ISP, chaque interface est minimaliste et ciblée.



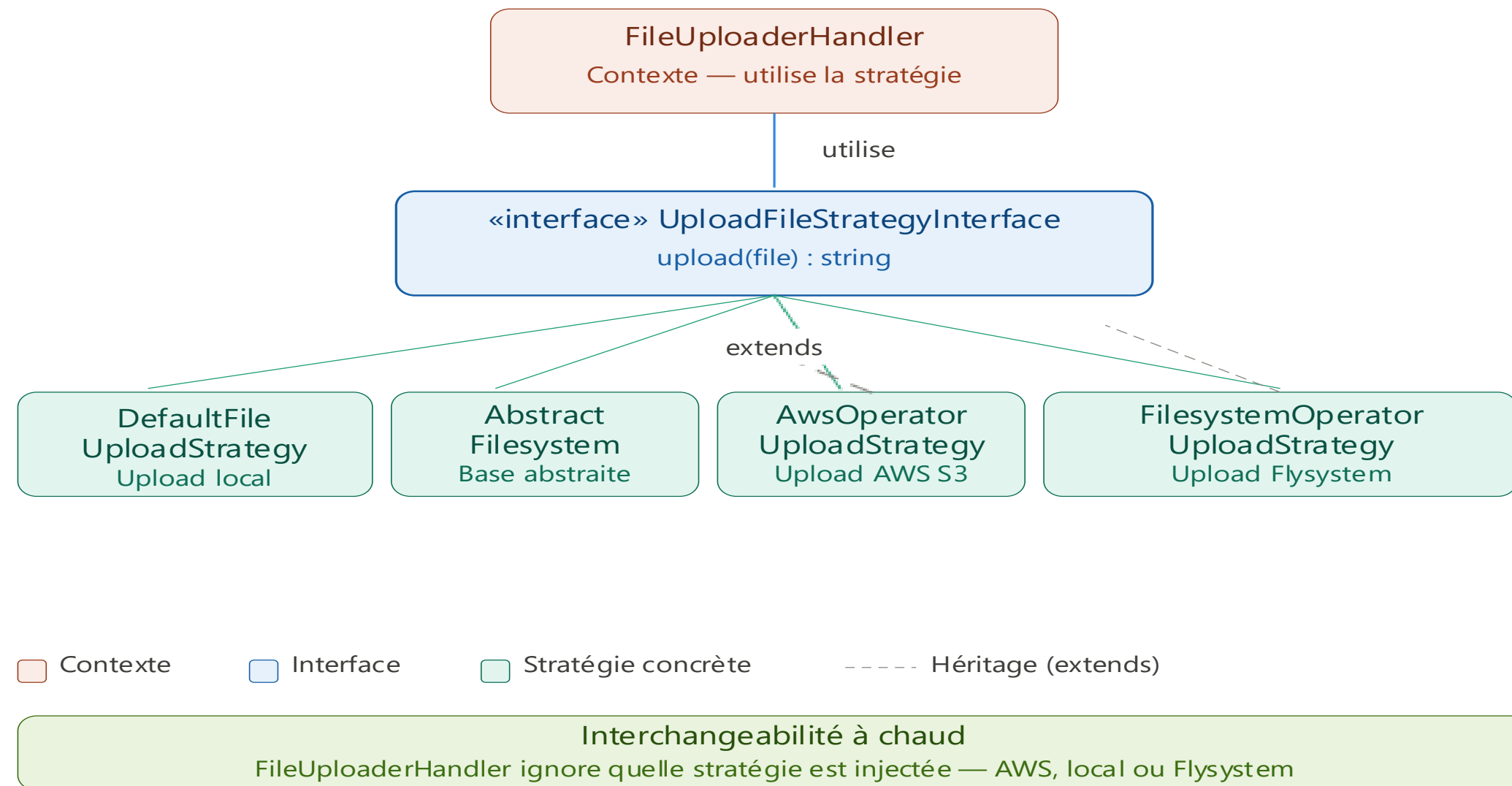
Besoins *Non Fonctionnels* (Principe *SOLID*)

D — Dependency Inversion : sans DIP, AuthController instancie directement new AuthHandler() — couplage fort, impossible à mocker. Avec DIP, tout passe par les interfaces enregistrées dans Program.cs, et le container ASP.NET Core injecte la bonne implémentation automatiquement.



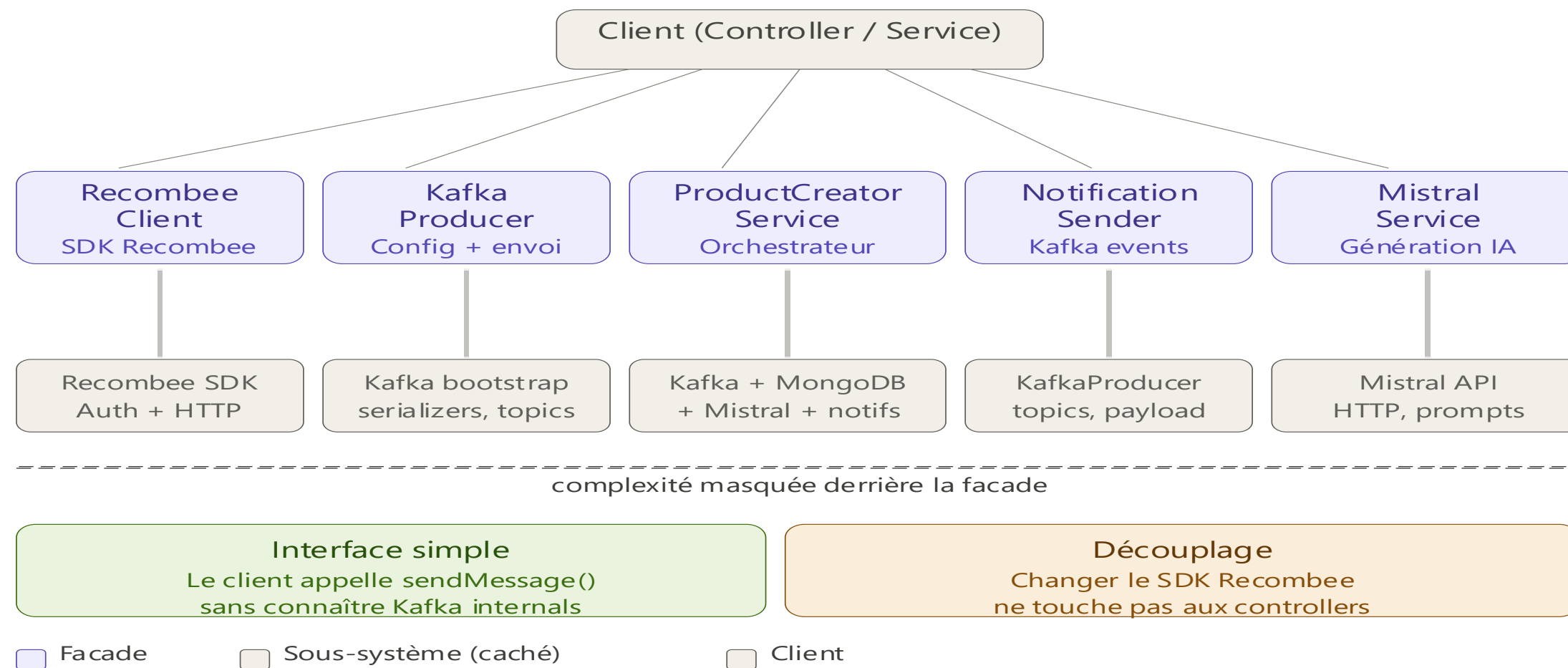
Besoins *Non Fonctionnels* (design patterns)

Le Strategy : FileUploaderHandler dépend uniquement de l'interface — on peut swapper la stratégie sans toucher au contexte. Voici maintenant le Facade Pattern.



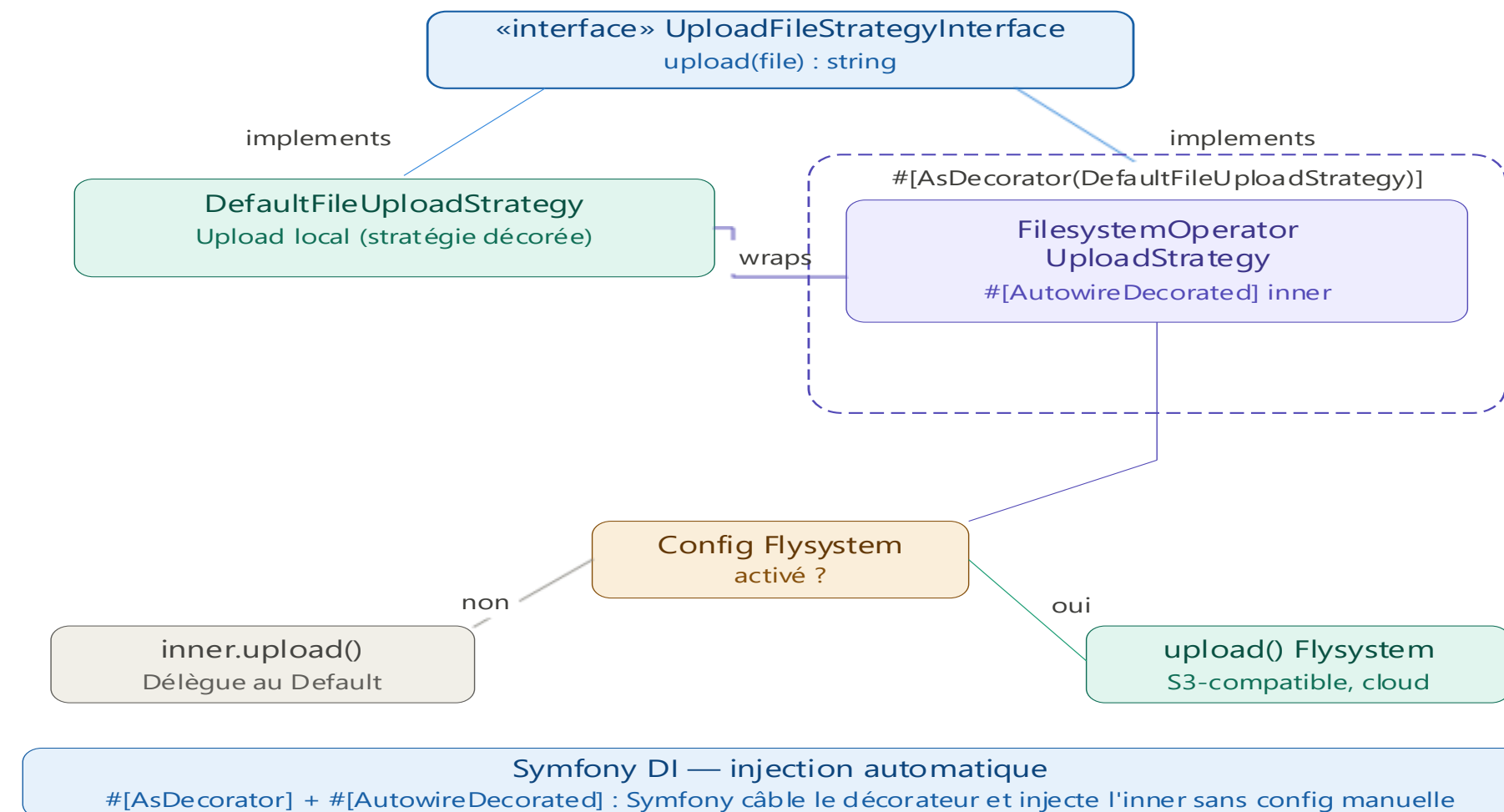
Besoins *Non Fonctionnels* (design patterns)

Facade — Client → 5 facades (violet) → sous-systèmes réels (Recombee, Kafka, MongoDB, Mistral AI). Chaque facade masque la complexité du système qu'elle enveloppe



Besoins *Non Fonctionnels* (design patterns)

Decorator — 3 colonnes : attributs Symfony ([AsDecorator] / [AutowireDecorated]) → couche emboîtée (amber entoure teal) → comportement runtime conditionnel avec losange de décision. Les nœuds sont cliquables pour approfondir.



04

Analyse & *Modélisation*

Diagramme *Use Case*

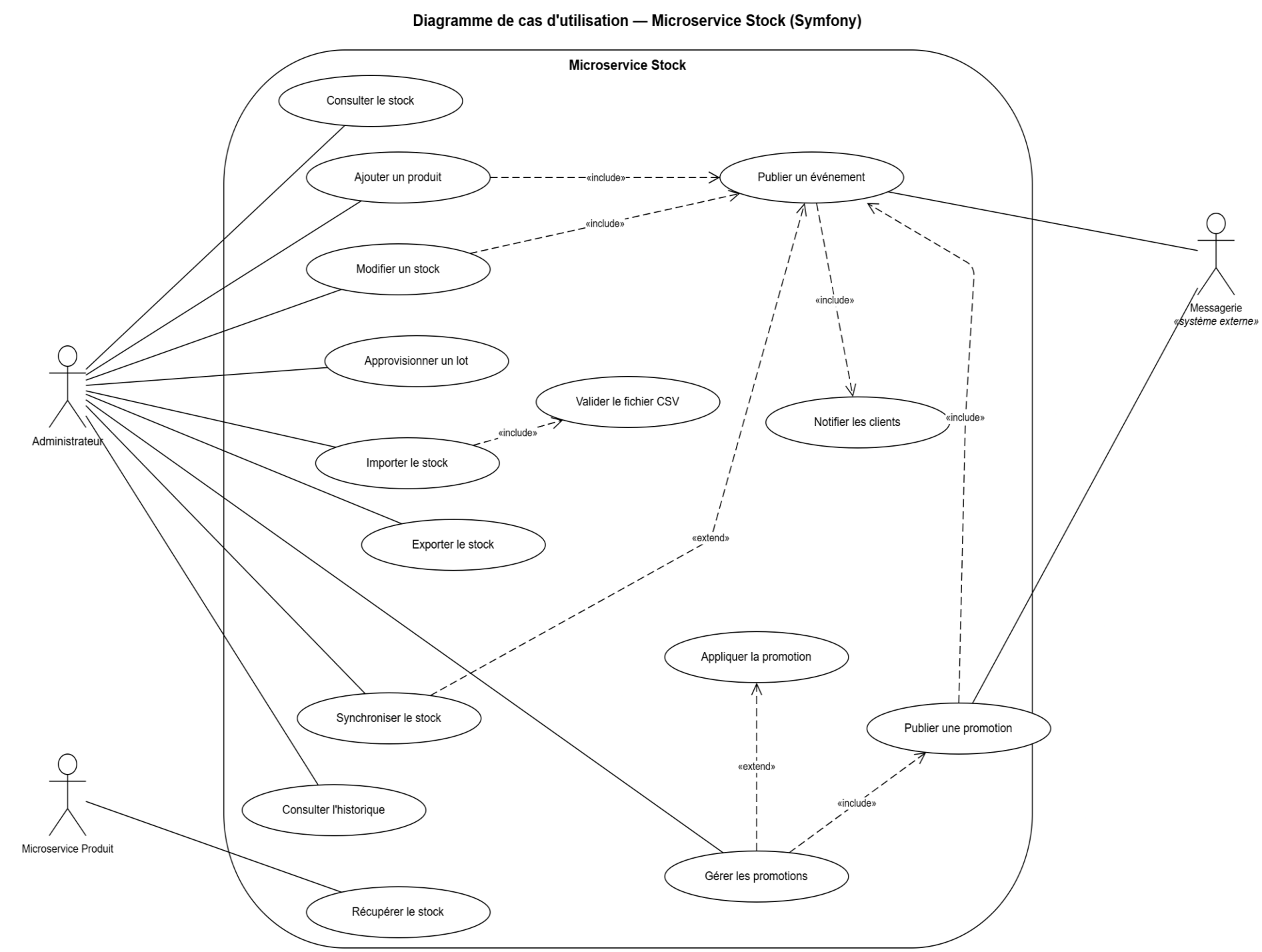


Diagramme de Séquence

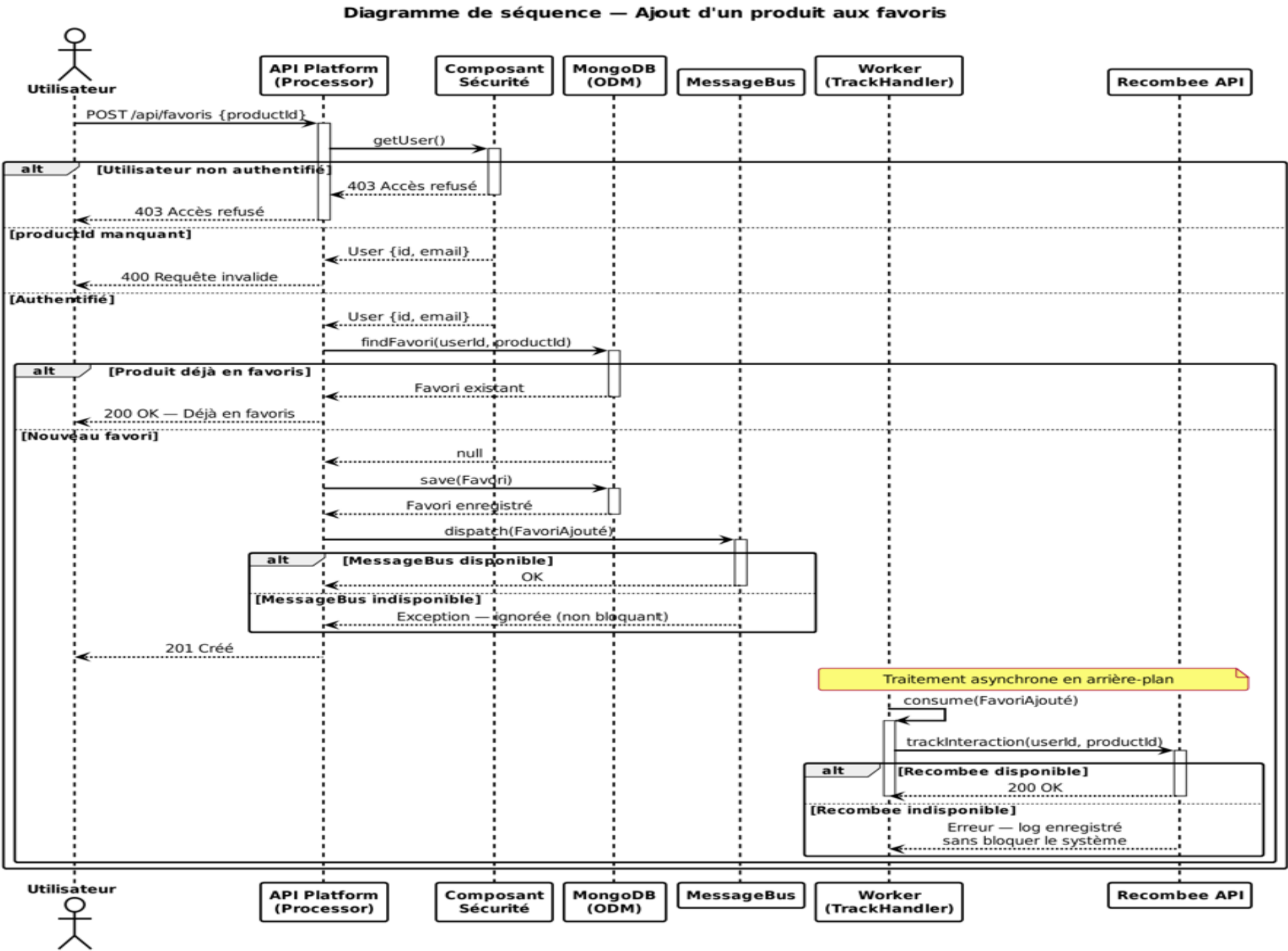
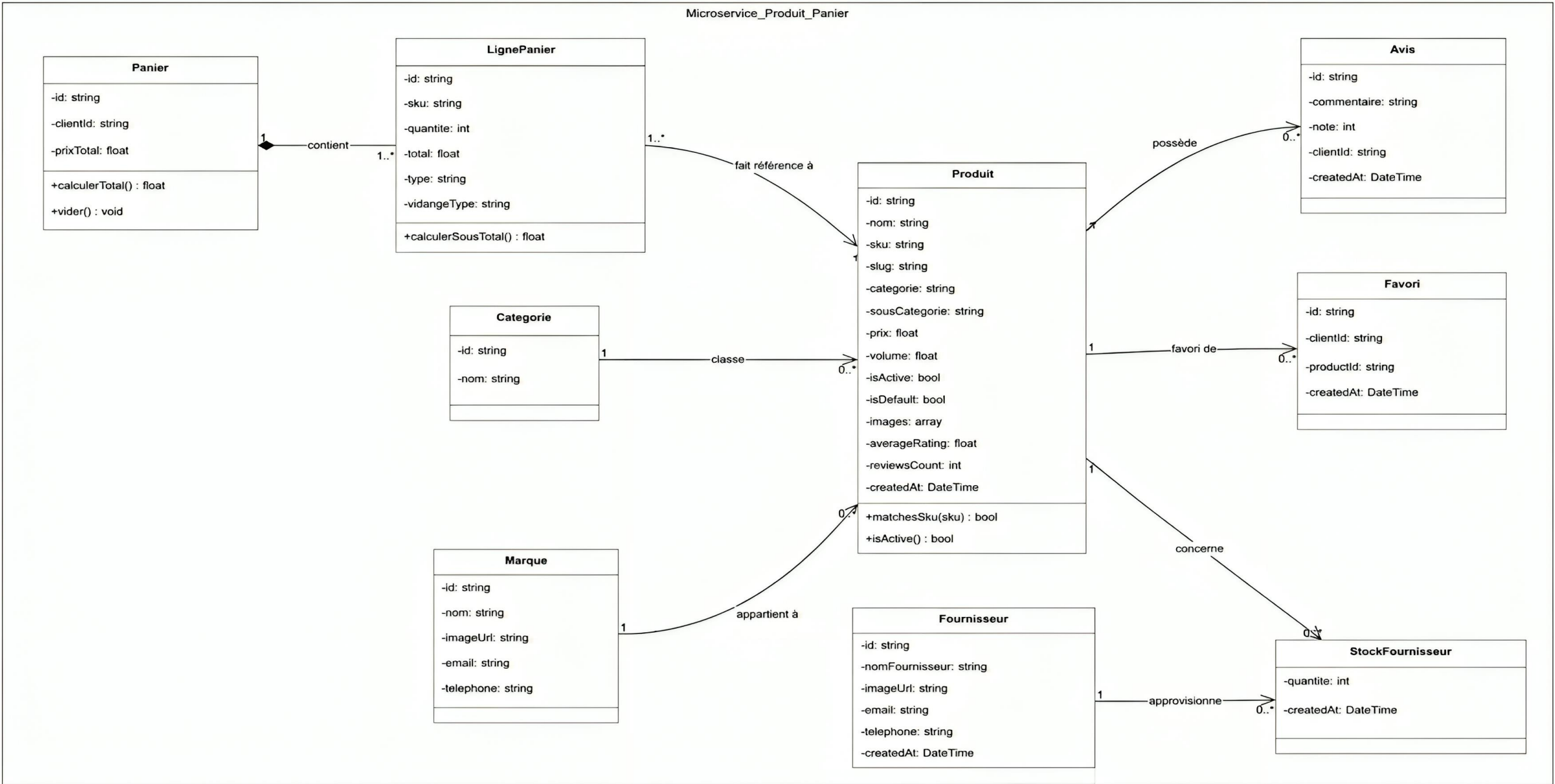


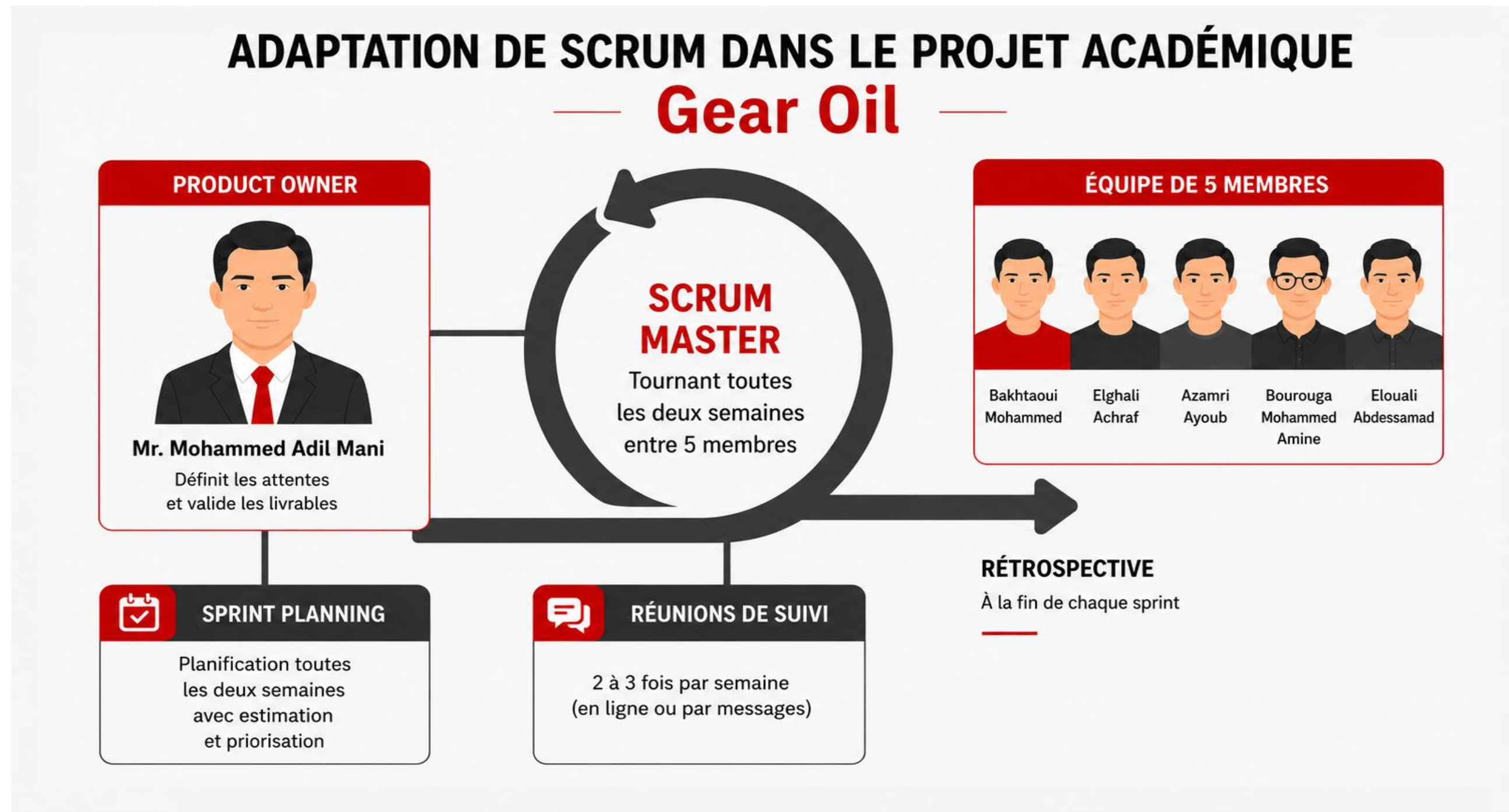
Diagramme *de Classes*



05

Réalisation *du Projet*

Scrum



Stack *Technique*

FRONTEND



Angular 21

-  • TypeScript 5.9
-  • Tailwind 3.4
-  • SCSS 1.86.3

IDENTITY - GATEWAY



.NET 9

-  • ASP.NET Core 9
-  • Ocelot 24.1 API Gateway
-  • EF Core 9
-  • SQL Server
-  • JWT Bearer

PAIEMENT - MÉTIER



Spring Boot 3.2.5

-  • Java 17 - MySQL 
-  • Stripe - Twilio 
-  • Spring Kafka - WebFlux 

BACKEND PRINCIPAL



Symfony 7

-  • PHP 8.2 - API Platform 4.3
-  • MongoDB -  AWS S3 
-  • Recommandée IA

MICROSERVICE STOCK



Symfony 7

-  • PHP 8.2 - API Platform 4.2
-  • Doctrine ORM - SQL 

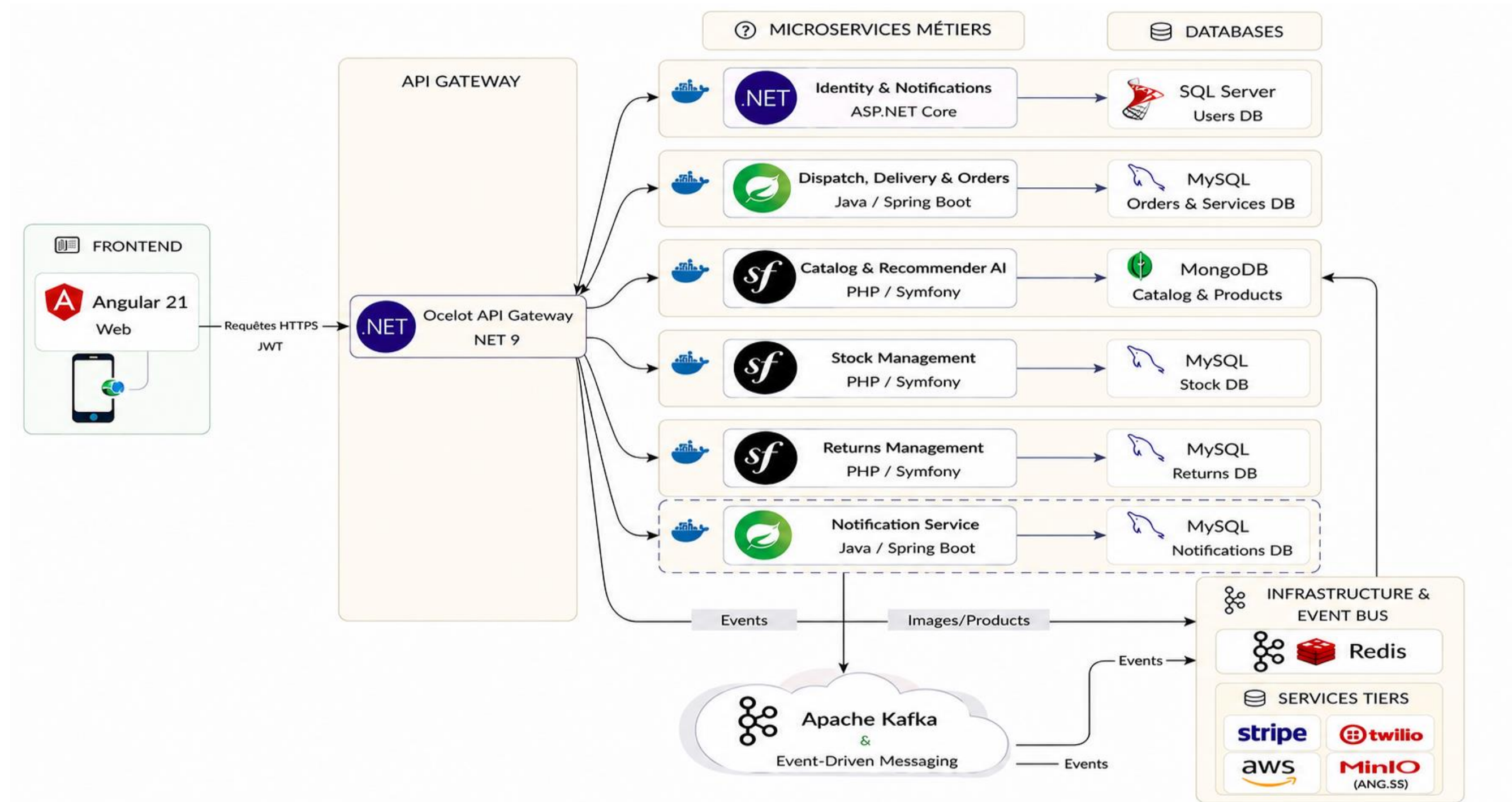
OUTILS - CI/CD



Docker + Git

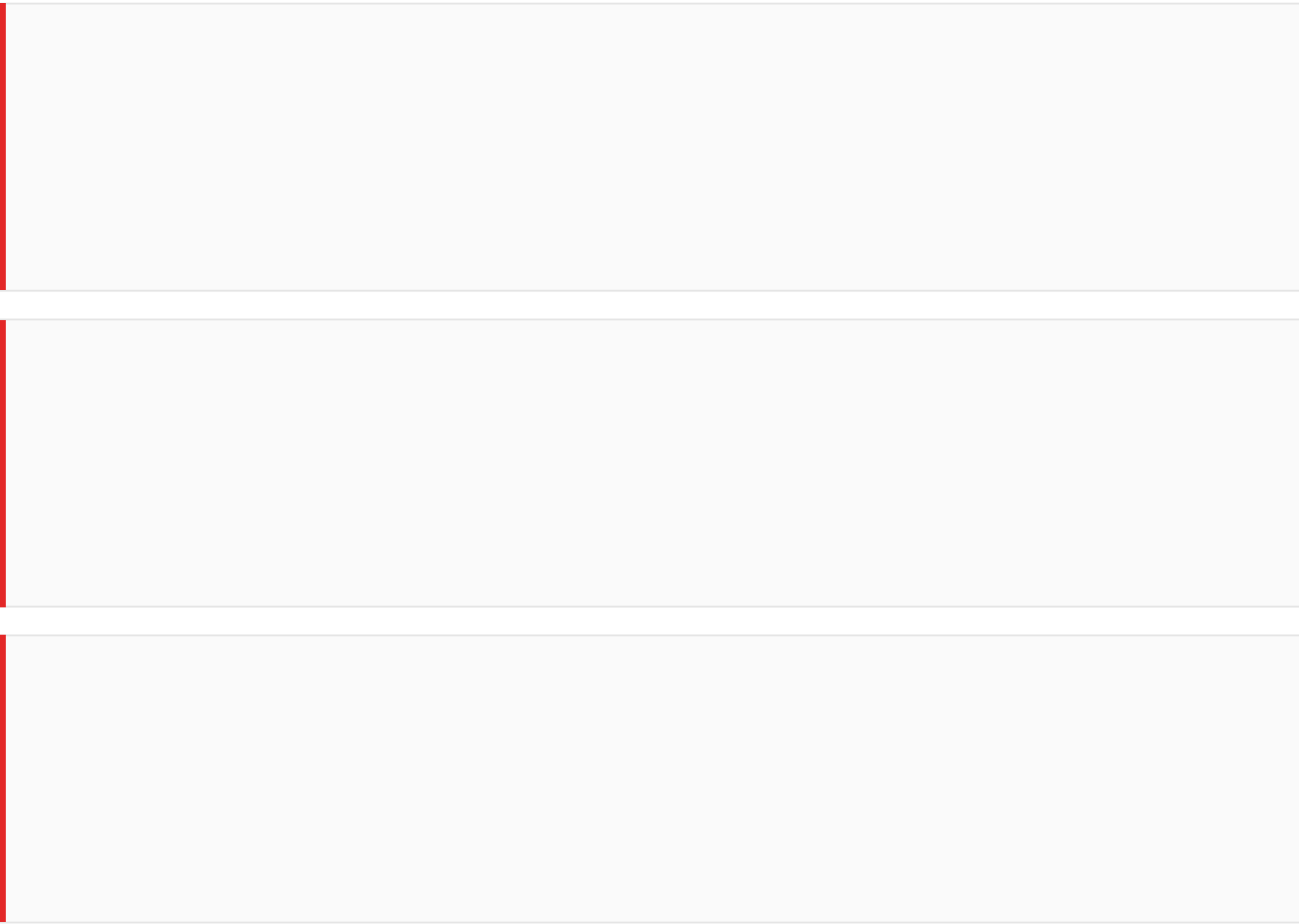
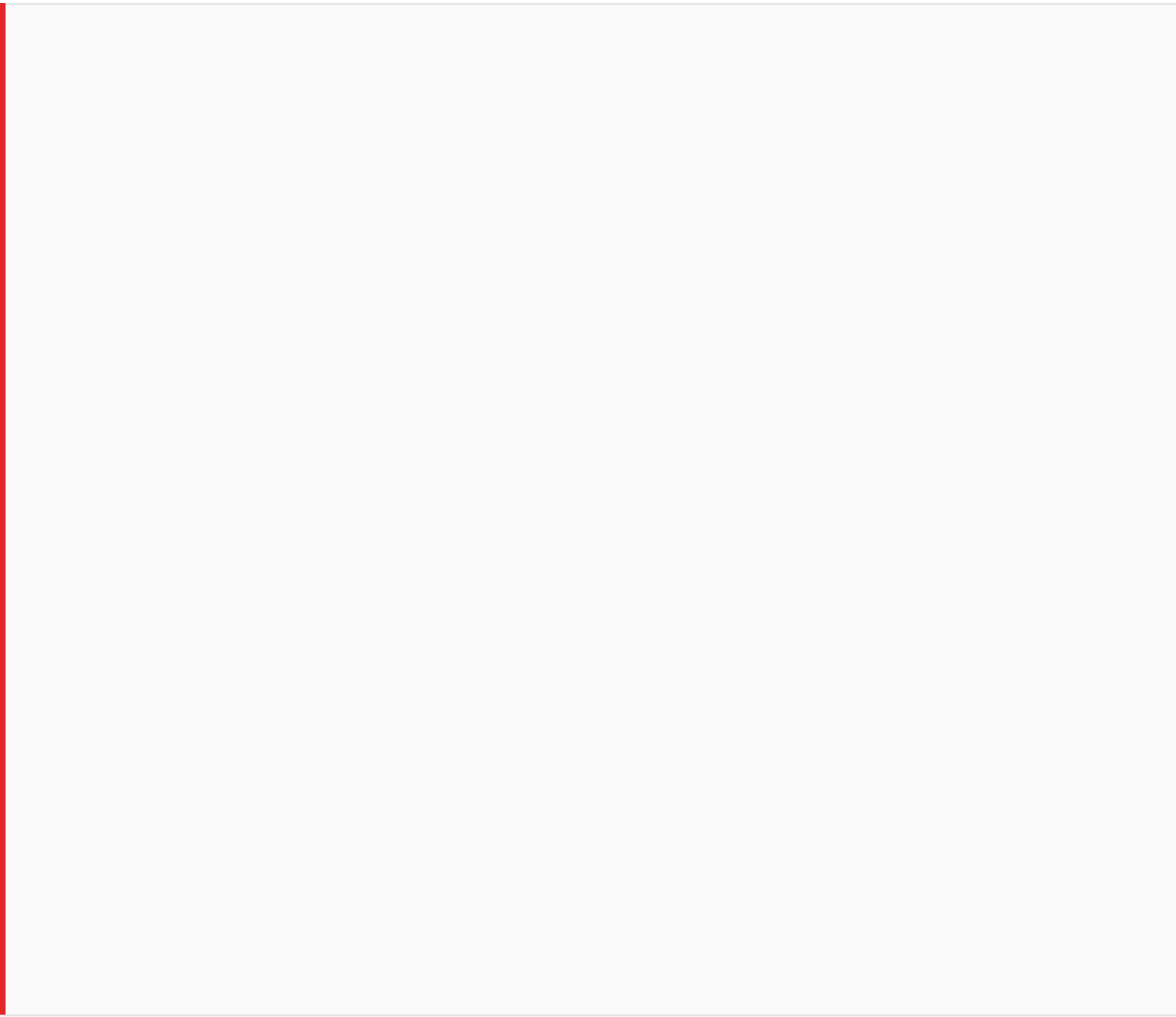
-  • Docker Compose - GitLab - GitFlow
-  • Postman - Swagger / OpenAPI
-  • JIRA - Scrum 2w

Architecture *Microservices*



DEMO

Perspectives



CONCLUSION



***MERCI POUR
VOTRE
ATTENTION***